

PHY604 Lecture 26

November 25, 2025

Today's lecture:

Neural networks and Parallel Computing

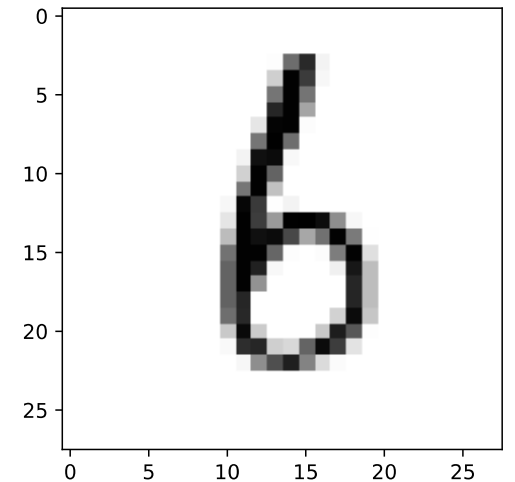
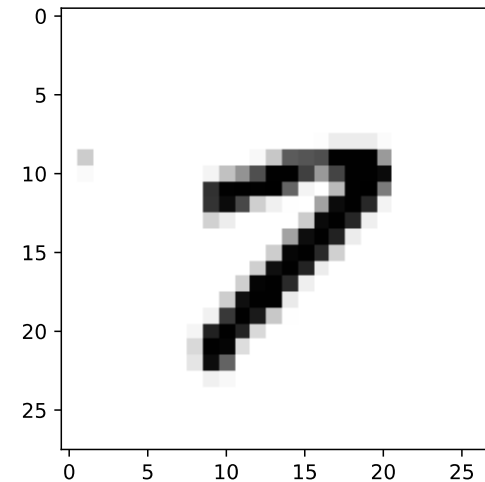
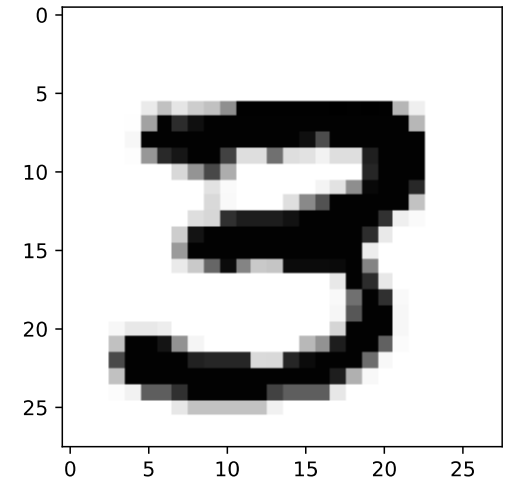
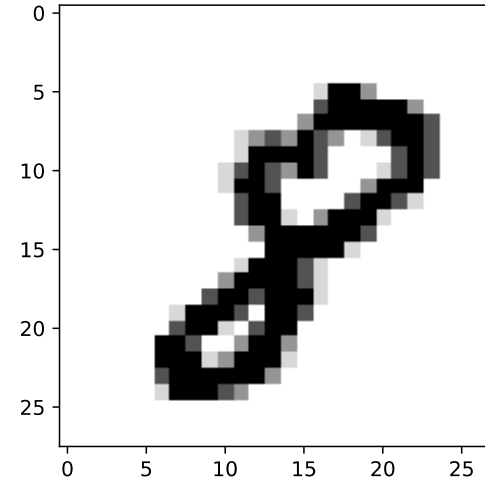
- Neural network example: recognizing characters
- Parallel computing

Another example: Recognizing written characters

- We'll try to recognize a digit (0 – 9) from an image of a handwritten digit.
- MNIST dataset (<http://yann.lecun.com/exdb/mnist/>)
 - Popular dataset for testing out machine learning techniques
 - Training set is 60,000 images
 - Approximately 250 different writers
 - Test set is 10,000 images
 - Correct answer is known for both sets so we can test our performance
- Image details:
 - 28×28 pixels, grayscale (0 – 255 intensity)
- We'll use a small subset

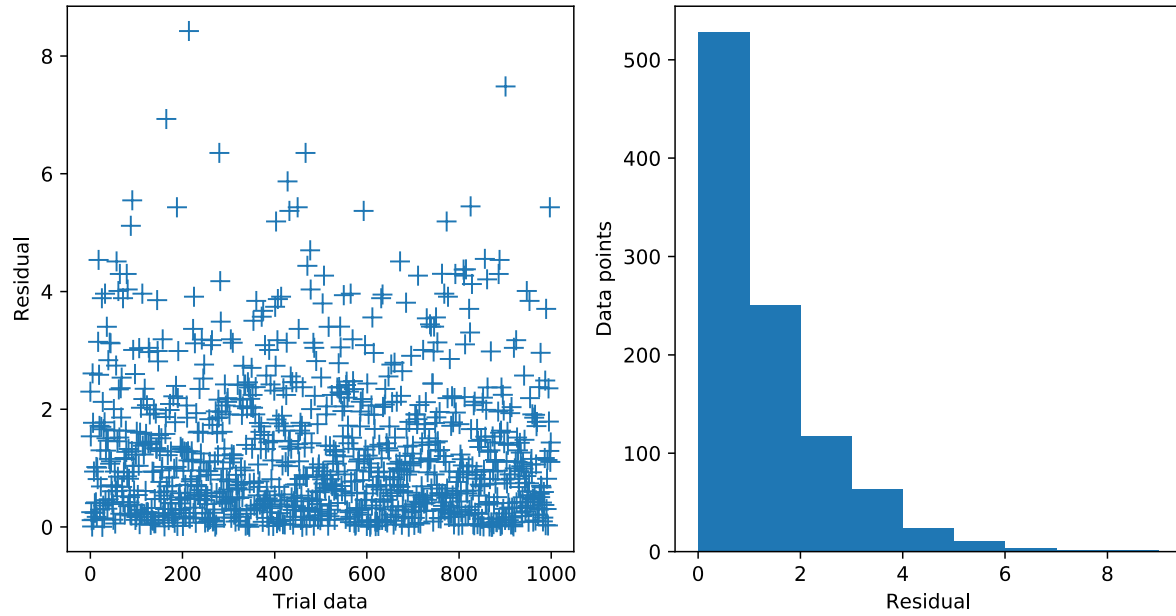
Another example: Recognizing written characters

- Input layer: 784 nodes (number of pixels)
- Output layer will be 10 nodes
 - Array with an entry for each possible digit
- Hidden layer size of 100
- 10 epochs
- We'll train on the training set, using 1000 images
- Rescale the input to be in $[0.01, 1]$
- We'll test on the test set of 1000 images

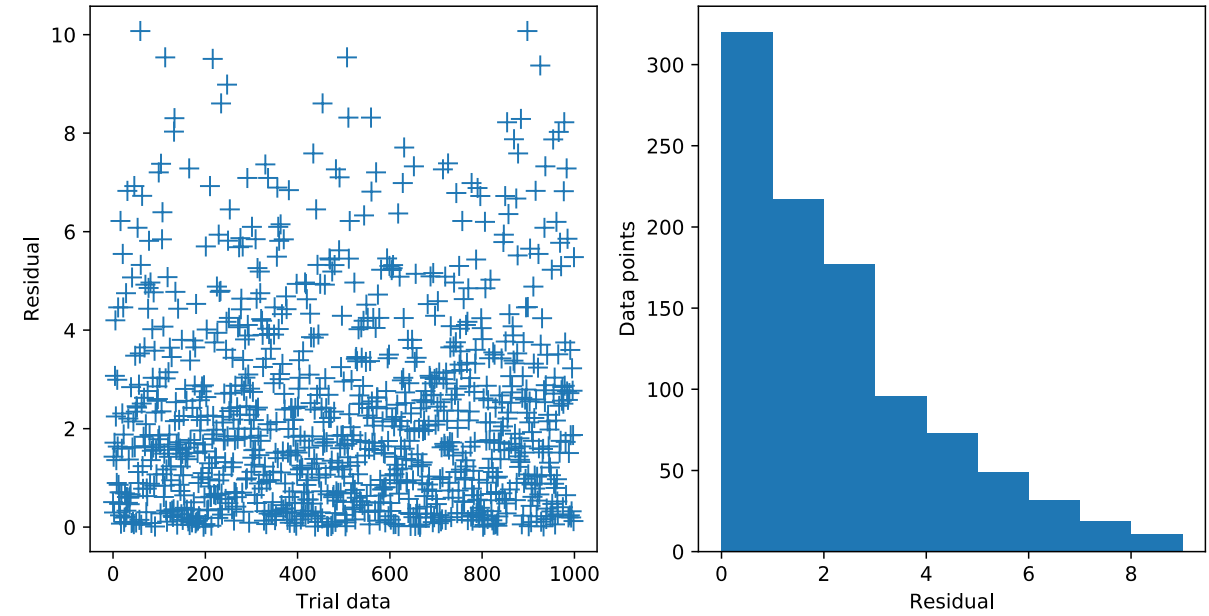


Another example: Recognizing written characters

Test on the training set



Test on new data



Today's lecture:

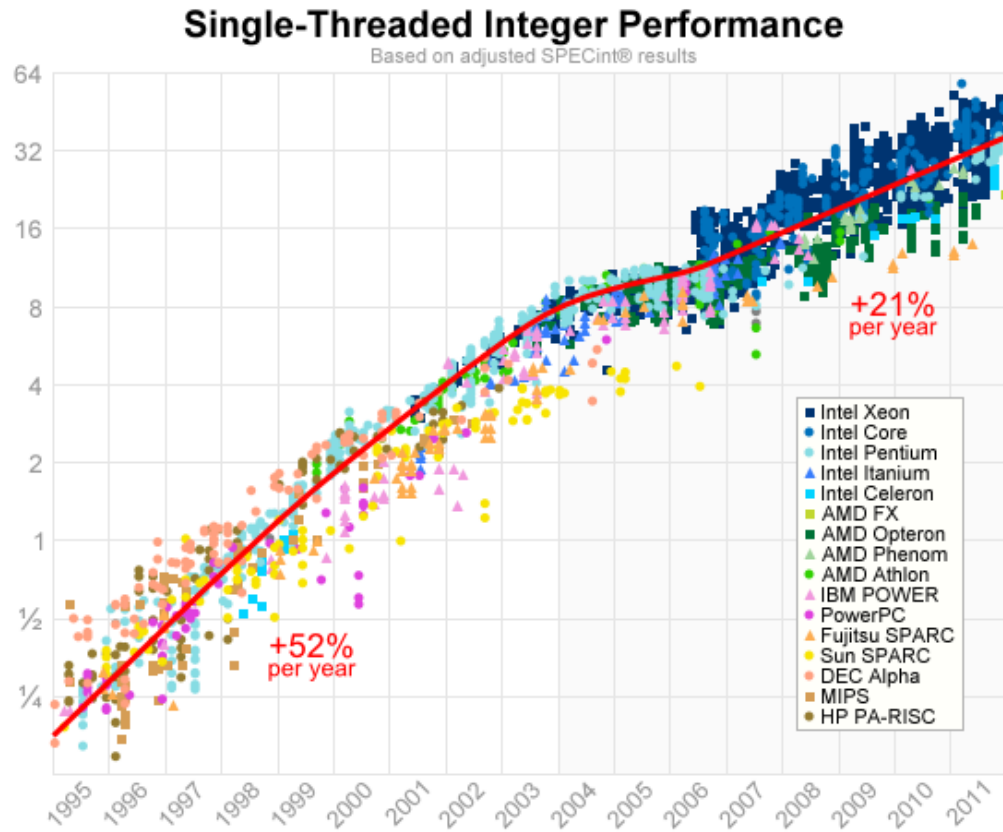
Neural networks and Parallel Computing

- Neural network example: recognizing characters
- Parallel computing

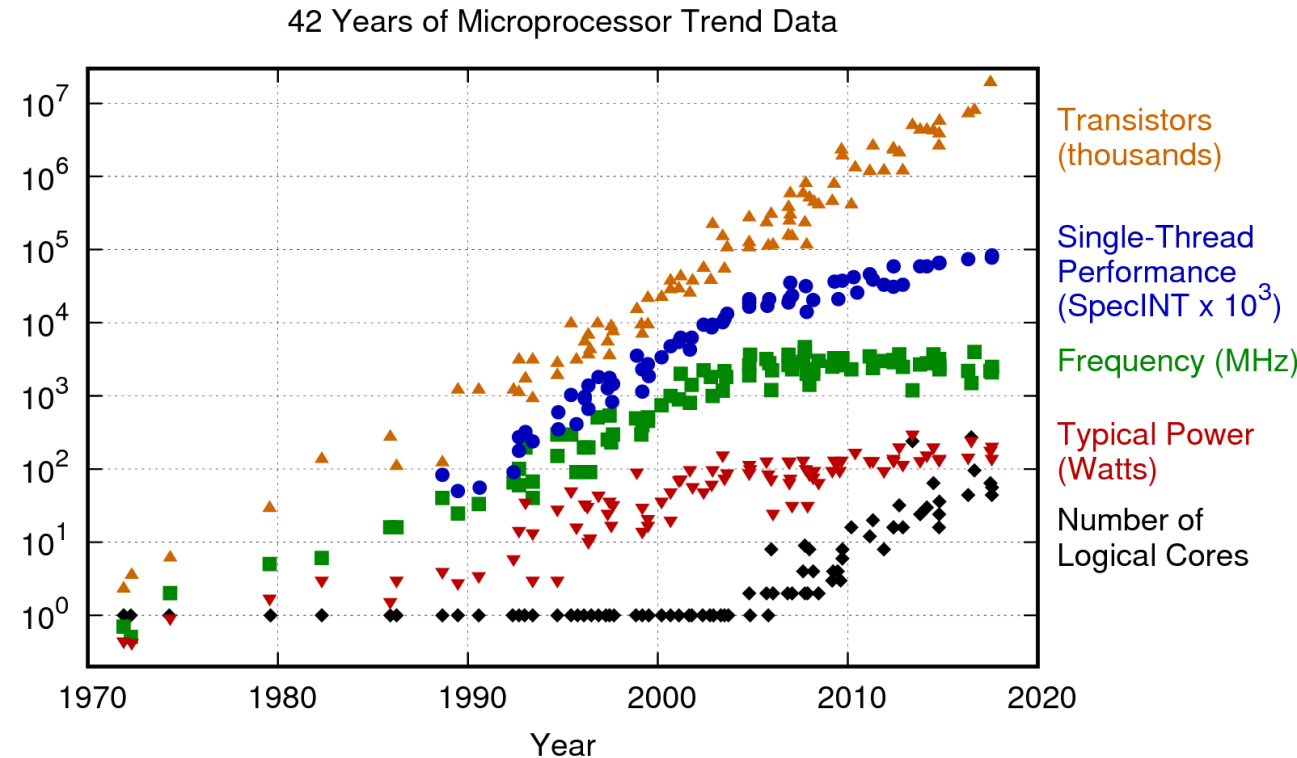
Why do we care about high performance computing?

- The more you can compute, the more interesting physics problems you can address
- Algorithmic advances allow us to do more with the same resources
 - This is what we have discussed over the course of the semester
- Another important aspect: Effectively taking advantage of more powerful computing hardware, e.g., supercomputers, GPUs

Performance of CPUs

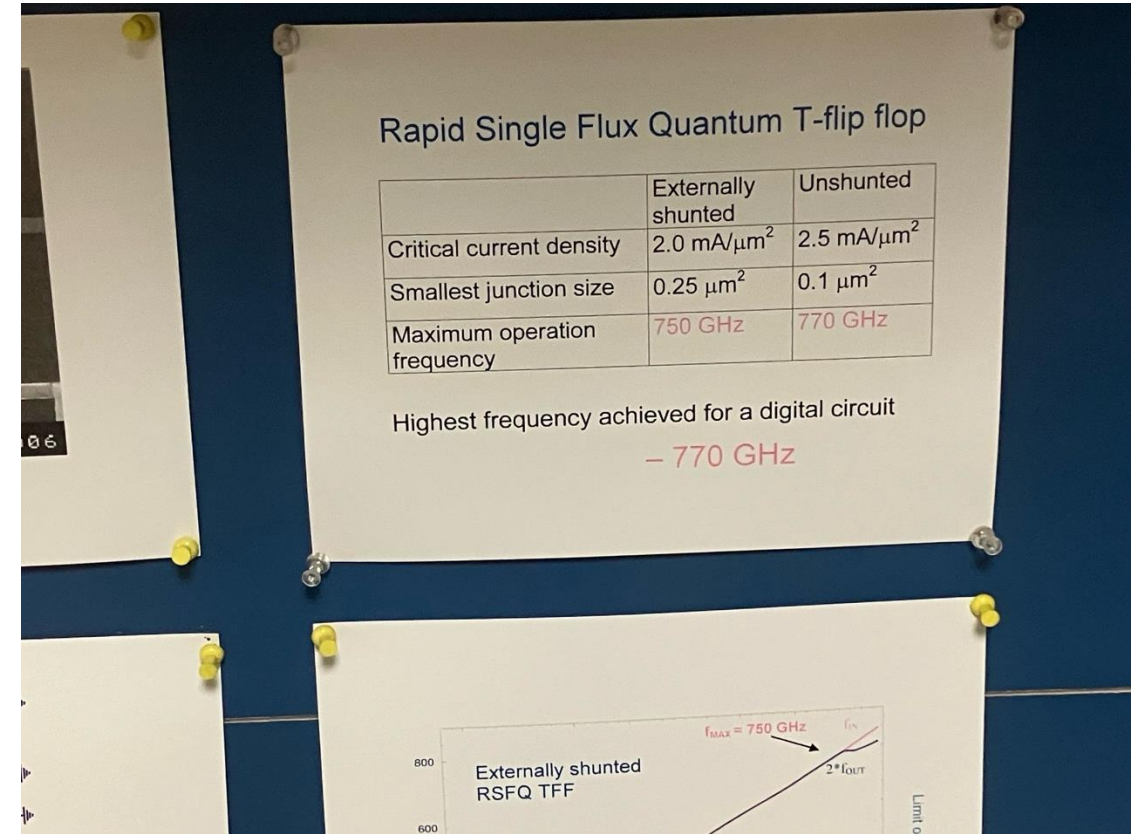
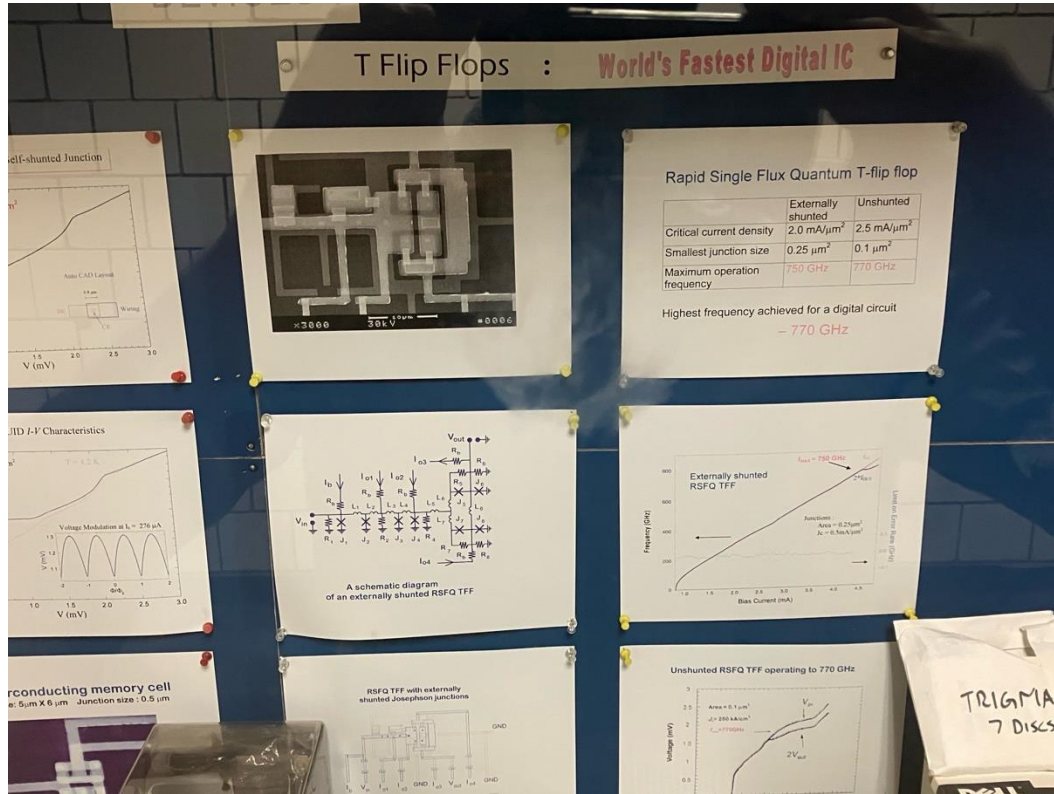


<https://preshing.com/20120208/a-look-back-at-single-threaded-cpu-performance/>



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2017 by K. Rupp

How can we do better? Superconducting electronics



- Record for clock speed: 770 GHz
- Normal CPUs 3-10 GHz

How can we do better? Parallel computing

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,039,616	1,742.00	2,746.38	29,581
2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	793.40	930.00	13,088
5	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure United States	2,073,600	561.20	846.84	

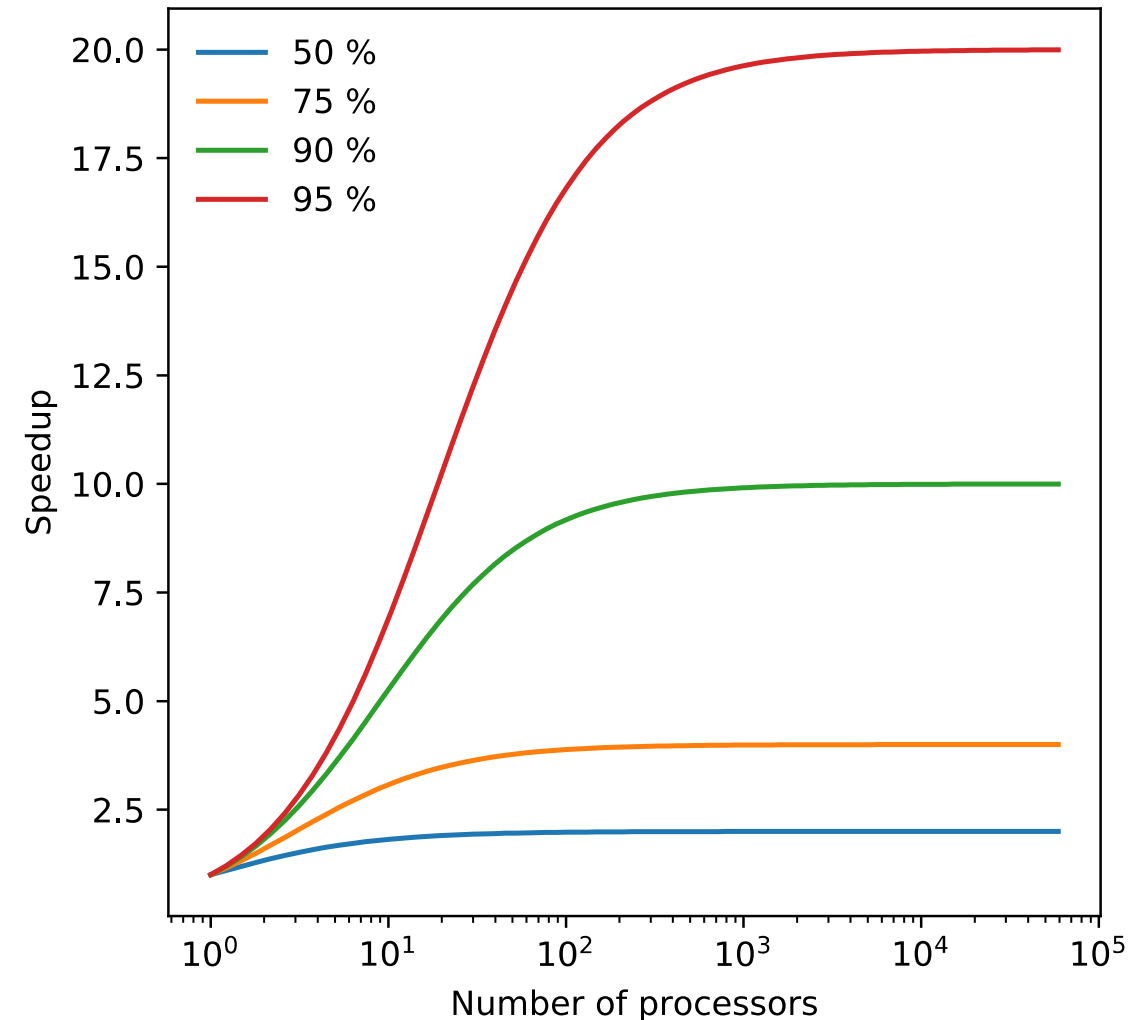
How do we write parallel programs?

- Determine what parts of the program are limiting
 - Profilers can help
- Determine the type of hardware you will be running your program on
- Decide how to parallelize limiting parts
- Test the scaling of your program with number of parallel processes

How parallel does the program need to be?

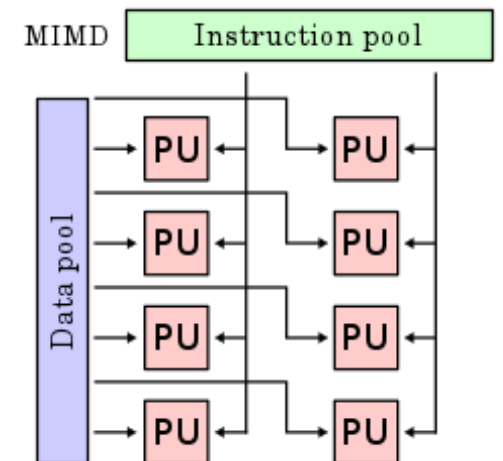
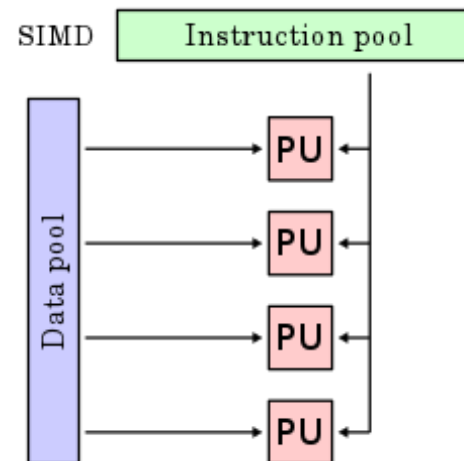
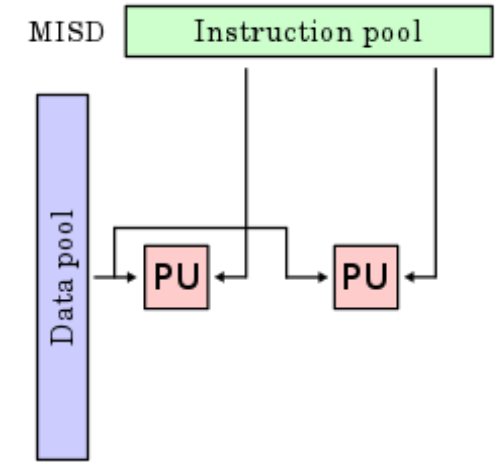
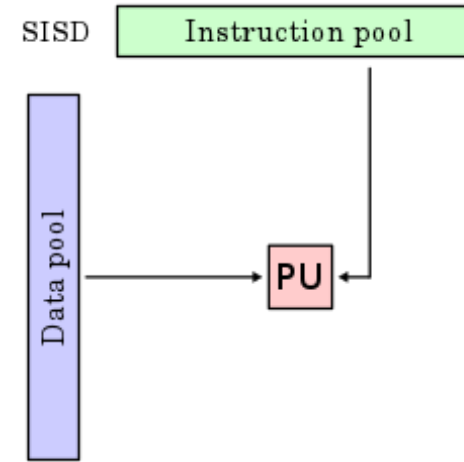
- The speedup depends on what portion of the program is parallel
 - Often, certain algorithms or parts of algorithms are not easy to parallelize
 - Not very useful if parallel parts are not the limiting parts
- **Amdahl's law**: speedup attained from increasing the number of processors, N , given the fraction of the code that is parallel, P :

$$S = \frac{1}{(1 - P) + (P/N)}$$



Types of parallelism: Flynn's taxonomy: ([wikipedia](https://en.wikipedia.org/wiki/Flynn's_taxonomy))

- Classification based on how the CPU receives instructions and data
- **Single instruction stream, single data stream (SISD)**
 - No parallelism, instructions and data accessed serially
- **Single instruction stream, multiple data streams (SIMD)**
 - Single instruction applied to multiple data streams
 - Structure of old vector processors, modern GPUs
- **Multiple instruction streams, single data stream (MISD)**
 - Multiple instructions applied to one data stream
 - Not very common
- **Multiple instruction streams, multiple data streams (MIMD)**
 - Multiple processors simultaneously executing different instructions on different data
 - This is what we normally consider parallel computing

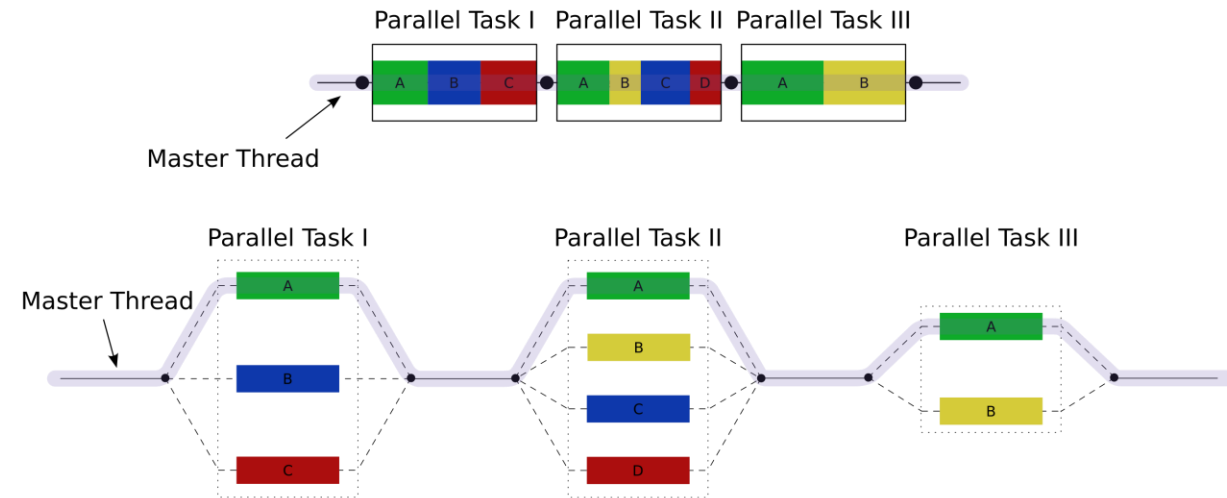


MIMD versus SPMD versus SIMD

- Normally, we do a specific type of MIMD:
 - Run a single program on multiple pieces of data (SPMD)
 - Different than SIMD since it does not require specialized hardware and does not require the processes to be in sync
- Two approaches to SPMD:
 - Distributed memory:
 - Machine has separate nodes which are essentially their own computer
 - Data (messages) are passed between nodes with interconnects, including commands to synchronize
 - Usually use **MPI** standard message passing interface (next time)
 - Shared memory:
 - Several CPUs have access to the same memory
 - Messages sent via the shared memory
 - Program starts executing on one processor, execution splits in a parallel region, then resynchronized
 - This is done via **OpenMP** (today)

OpenMP and multithreading

- Start process on a master thread
- Each time a piece of code is reached that is parallelized, the master thread forks subthreads to run the task
- After the parallel task is completed, threads join back to the master



Wikipedia

Examples of OpenMP

- To use OpenMP, we modify our program with directives or pragmas
 - Look like comments, but tells the compiler how it should process the input
- We also need to compile with flags for OpenMP
 - E.g., `gfortran -fopenmp`
- Finally, environment variables can also set properties
 - `OMP_NUM_THREADS` sets the number of threads
- Simplest example:

```
program hello

    !$OMP parallel
    print *, "Hello world"
    !$OMP end parallel

end program hello
```


OMP functions

- OMP provides some functions to get the number of threads, current thread, etc.:

```
program hello

  use omp_lib

  write(*,*) "outside of parallel region, num threads = ", omp_get_num_threads()

  !$OMP parallel
  write(*,*) "Hello world", omp_get_thread_num()
  !$OMP end parallel

end program hello
```

How can we get a speedup from threading?

- In the last example, we just did the same thing multiple times
- Simplest code to nontrivially parallelize are loops:

```
!$omp parallel private(i, j)
!$omp do
do j = 1,N
    do i = 1,N
        a(i,j)=0.
    end do
end do
!$omp end do
!$omp end parallel
```

Parallel calculations with OpenMP

- Can set the number of threads available with `OMP_NUM_THREADS`
 - Your code will most likely still run if you ask for more threads than cores, but you will not get a speedup!
- There is some overhead for spawning the threads
- As discussed before, best to parallelize limiting parts of the program first

Private versus shared data

- Inside the parallel tasks, can specify if each thread has it's own copy (private) or not (shared)
 - Shared is the default for variables declared outside of the parallel task
 - Be careful modifying shared data!
- Consider the two pieces of code:

```
k=0
!$omp parallel shared(k)
k=omp_get_thread_num()
!$omp end parallel
write (*,*) 'k=', k
```

```
k=0
!$omp parallel private(k)
k=omp_get_thread_num()
!$omp end parallel
write (*,*) 'k=', k
```

- The first one shares k over the nodes, the second makes private copies on all nodes

Combining data with reduction

- We can combine the results from all threads automatically via reduction:

```
k=0
!$omp parallel reduction(+:k)
k=omp_get_thread_num()
!$omp end parallel
write(*,*) 'k=',k
```

- Will add the results of each thread

Controlling ordering a synchronization of threads

- As we saw before the threads will proceed in some unknown order
- We may want to have more control than this
- Some useful directives:
 - `!$omp critical`: Make sure only one thread at a time can changes things in critical sections
 - Creates a “thread safe” section of code
 - Important when, e.g., incrementing shared variables
 - `!$omp barrier`: Stops processes from proceeding until all processes catch up

Some comments on OpenMP

- We just covered some basics, OpenMP has quite a bit of functionality
- Have to make sure that your program is threadsafe:
 - Don't inadvertently overwrite data
 - Don't have issues because of the order or synchronization of processes
 - Always good unit test in serial and parallel to make sure the results are the same
- Fairly straightforward to parallelize your code piece by piece
 - Because directives look like comments, it is easy to run with and without parallelization (just change compiler flags)
- Works well for fortran, C, C++
- Will NOT work for pure python
 - "Global interpreter lock" that means only one thread can talk to the interpreter at any one time
 - Can use OpenMP with, e.g., Cython

After class tasks

- Homework 5 graded
- Have a nice Thanksgiving!